

# 머신러닝 2 교시

## 데이터 탐색과 전처리 (Exploratory Data Analysis · Preprocessing)

### 학습 목표

- 머신러닝에서 데이터 전처리가 왜 필요한지 설명할 수 있다.
- `describe()`, `info()`, `value_counts()` 등 pandas 기본 기능으로 데이터 탐색(EDA)을 수행할 수 있다.
- 결측치(NaN)를 발견하고 행 삭제 / 평균·중앙값·최빈값 대체 / 그룹별 대체를 상황에 맞게 적용할 수 있다.
- 박스플롯과 IQR 규칙으로 이상치를 탐지하고 처리 방침을 결정할 수 있다.
- 표준화(StandardScaler)와 정규화(MinMaxScaler)의 차이와 적용 상황을 안다.
- 범주형 변수를 Label Encoding, One-Hot Encoding 으로 변환할 수 있다.
- Titanic 데이터에 전처리를 적용한 뒤 KNN 정확도가 어떻게 달라지는지 직접 비교할 수 있다.

### 1. 왜 전처리(Preprocessing)가 필요한가

앞에서 우리는 사이킷런이 깔끔하게 정리해 둔 iris 와 breast\_cancer 데이터를 사용했다. 그러나 실제 현장의 데이터는 다음과 같은 상태로 들어온다.

- 측정값이 비어 있는 칸(결측치)이 곳곳에 있다.
- 일부 값이 비정상적으로 크거나 작은 이상치(outlier)다.
- 값의 단위와 스케일이 들쭉날쭉하다 — 예: 나이(0~80)와 운임(0~512\$).
- 문자열로 된 범주형 변수(성별, 도시명 등)는 모델이 직접 다룰 수 없다.

이런 "날 데이터(raw data)"를 그대로 모델에 넣으면 학습이 실패하거나, 학습이 되더라도 성능이 형편없다. 데이터 전처리는 "모델을 잘 학습시키기 위한 모든 준비 작업"을 가리킨다. 흔히 "머신러닝 프로젝트 시간의 70~80%가 전처리에 들어간다"고 말할 만큼 비중이 크다.

## 전처리 작업을 한눈에

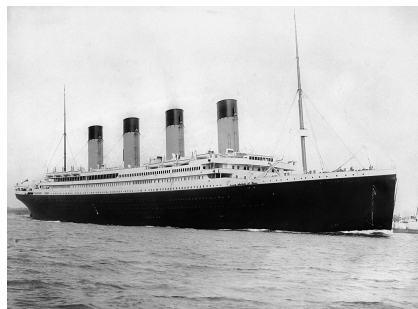
전처리 큰 그림

- └── EDA (탐색적 데이터 분석)
  - | └── 데이터 모양·자료형 점검
  - | └── 분포·상관관계 시각화
  - | └── 클래스 균형 확인
- └── 결측치 처리
  - | └── 행/열 삭제
  - | └── 대체(평균·중앙값·최빈값·모델 기반)
- └── 이상치 처리
  - | └── 검출(IQR, z-score)
  - | └── 제거 또는 윈저화(winsorize)
- └── 스케일링
  - | └── 표준화 (StandardScaler)
  - | └── 정규화 (MinMaxScaler)
- └── 범주형 인코딩
  - | └── Label Encoding
  - | └── One-Hot Encoding

## 2. 데이터 탐색 (EDA)

EDA(Exploratory Data Analysis)는 "이 데이터가 어떻게 생겼는가"를 처음부터 끝까지 파악하는 단계다. 모델 선택과 전처리 전략은 EDA 에서 발견한 "이 데이터의 특성"에 의해 결정된다.

*Titanic 데이터셋은 결측치·이상치·범주형 변수가 모두 섞여 있어 전처리 학습용 "교과서" 데이터다.*



### 2.1 pandas 로 살펴보기

```
import pandas as pd, seaborn as sns
df = sns.load_dataset("titanic")
```

```
print(df.shape)      # (891, 15) — 행/열 개수
print(df.dtypes)     # 자료형
print(df.head())     # 앞 5 행
print(df.describe()) # 수치형 요약 통계 (count, mean, std, min, 25%, 50%, 75%, max)
print(df.info())     # 결측치 개수 한눈에
```

describe()의 출력에서 max 값이 평균에서 멀리 벗어나 있거나, count 가 행 개수와 다르면(=결측치 존재) 즉시 다음 처리 단계가 무엇이 되어야 할지 보인다.

### 3. 결측치(Missing Value) : 개수확인 and 처리

결측치는 데이터셋의 어떤 칸이 비어 있는 상태(NaN, None)다. Titanic 데이터의 경우 age(나이)에 약 20%, deck(객실 구역)에 70% 이상의 결측치가 있다. 결측치는 그 자체로 정보일 때도 있고("이 컬럼은 측정 불가능했다"), 단순한 누락일 때도 있다.

#### 3.1 결측치 패턴 확인

```
# 컬럼별 결측치 비율
miss = df.isnull().mean().sort_values(ascending=False)
print(miss)

# 시각적 패턴
import matplotlib.pyplot as plt
import seaborn as sns
sns.heatmap(df.isnull(), cbar=False)
plt.show()

# 또는 다른 라이브러리를 쓸 수도 있다
import missingno as msno
msno.matrix(df) # 막대 그래프 형태의 msno.bar(df)도 유용함
```

히트맵에서 검은색 부분은 데이터가 정상적으로 존재하는 영역이고 흰색이 결측치이다. "가로 줄 무늬"로 보이면 특정 행 전체가 결측치라는 뜻이고 "세로 막대" 모양으로 결측치가 보이면 그 컬럼은 통째로 누락된 것이다.

#### 3.2 처리 전략 — 무엇이 적절한가

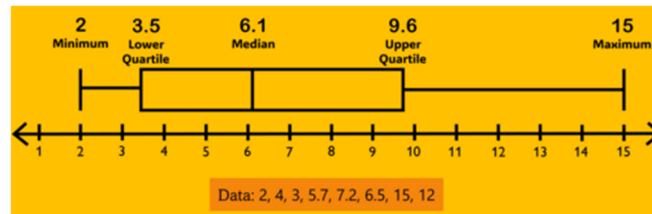
```
결측치 처리 의사결정
|—— 결측 비율 < 5% → 행 또는 열 삭제
|—— 결측 비율 5~30% → 대체(impute)
|   |—— 수치형 → 평균 / 중앙값(이상치에 강함)
|   |—— 범주형 → 최빈값
|   |—— 그룹별 대체(예: 성별별 평균 나이)
|—— 결측 비율 > 50% → 컬럼 자체 제거 검토
|—— 결측 자체가 정보일 가능성
|   — "missing"이라는 별도 카테고리 또는 IsMissing 더미 변수 추가
```

## 4. 이상치(Outlier): 탐지와 처리

이상치는 다른 관측값들과 동떨어진 값이다. 측정 오류일 수도 있고, 정말로 드문 사건일 수도 있다. 모델에 따라 영향이 크게 다르다 — 선형회귀·KNN 은 이상치에 매우 민감하지만, 트리 기반 모델은 비교적 강하다.

### 4.1 분포 시각화 — 박스플롯

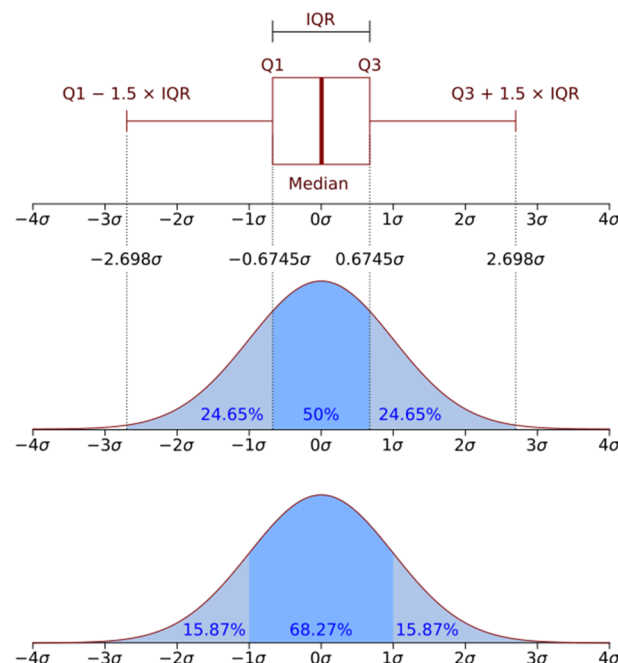
수치형 변수의 분포는 박스플롯으로 한눈에 본다. 박스플롯은 다섯 수치(최솟값·1 사분위·중앙값·3 사분위·최댓값)와 이상치를 한 그림에 담은 시각화다.



박스플롯의 구성. 상자(box)는 데이터의 가운데 50%( $IQR = Q3 - Q1$ )이고, 가운데 굵은 선이 중앙값. 양 끝의 수염(whisker)은  $IQR$  의 1.5 배까지 뻗으며, 이를 벗어나는 점은 이상치(outlier)로 표시된다.

### 4.2 박스플롯과 분포의 관계

박스플롯은 단순히 "선 5 개"를 긋는 것이 아니라, 분포 전체를 압축적으로 보여준다. 정규분포(좌우 대칭) 데이터의 경우 박스의 좌우 폭이 같고 수염도 거의 같다. 데이터가 한쪽으로 치우친(skewed) 경우 박스의 중앙선이 한쪽으로 쏠리고, 한쪽 수염이 더 길어진다.



[그림 2] 박스플롯과 확률밀도함수(PDF)의 대응. 정규분포에서는  $IQR(Q1 \sim Q3)$  이 분포의 중앙 ~50%를 차지하며, 수염 끝( $1.5 \times IQR$ )은 약  $\pm 2.7\sigma$  부근까지 뻗는다.

### 4.3 IQR 을 사용하여 이상치 찾기

```
# IQR 기반 이상치 검출
q1, q3 = df["fare"].quantile([0.25, 0.75])
iqr = q3 - q1
low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr

outliers = df[(df["fare"] < low) | (df["fare"] > high)]
print(f"이상치 비율: {len(outliers) / len(df):.2%}")
```

### 4.2 처리 방법

- 제거 — 명백한 측정 오류일 때만.
- 윈저화(winsorization) — 양 끝을 percentile 값으로 클리핑(예: 99% 분위수로).
- 로그 변환 — 운임처럼 한쪽으로 치우친 분포를 펴서 이상치 영향을 완화.
- 로버스트 스케일러 사용 — 평균·표준편차 대신 중앙값·IQR 기반으로 스케일링.

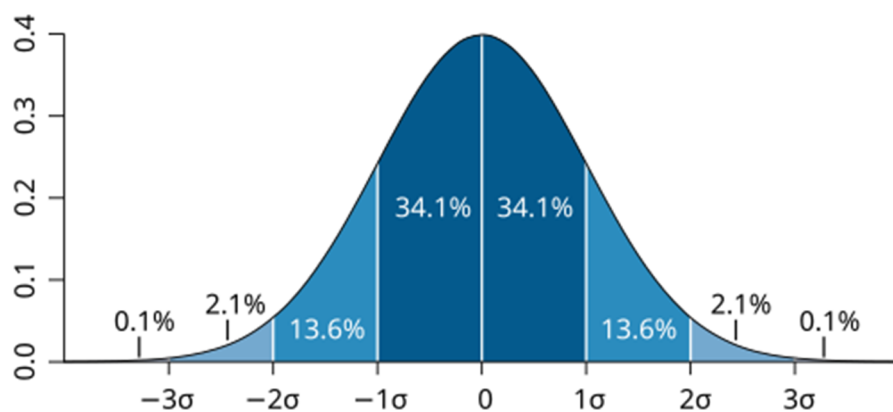
## 5. 스케일링(Scaling) : 표준화와 정규화

스케일링은 서로 다른 단위·범위를 가진 수치형 특성을 비슷한 스케일로 맞추는 작업이다. KNN, 로지스틱 회귀, SVM, k-means, 신경망 등 "거리 기반" 또는 "기울기 기반" 모델에서는 스케일링이 거의 필수다.

### 5.1 표준화 (StandardScaler)

z-score 를 이용해 평균 0, 표준편차 1 로 변환한다. 정규분포에 가까운 데이터에 가장 자연스럽다.

$$z = (x - \text{mean}) / \text{std}$$



[그림3] 표준화의 의미 — 표준정규분포에서  $\pm 1\sigma$  안에 약 68%,  $\pm 2\sigma$  안에 약 95%의 값이 들어간다. 표준화 후의 값은 "이 값이 평균에서 몇  $\sigma$  떨어져 있나"를 직관적으로 알려준다.

직접 수식을 만들어도 되고, scikit-learn 에서 제공하는 Standard Scaler 를 써도 된다.

객체를 만든다 -> . fit 한다 -> . transform 한다.

```
scaler = StandardScaler() # 1. 객체를 만든다
scaler.fit(df)            # 2. fit 한다
scaled = scaler.transform(df) # 3. transform 한다.
```

## 5.2 정규화 (MinMaxScaler)

최솟값과 최댓값으로 0~1 범위로 변환한다. 거리 합산이 의미를 가져야 하는 알고리즘(예: 신경망 입력)에 자주 쓰인다. 단, 이상치가 있으면 다른 모든 값이 좁은 범위에 몰린다.

$$x' = (x - \min) / (\max - \min)$$

`lambda x : (x - min(a)) / (max(a) - min(a))` 함수를 사용하여 `df.apply()`를 하면 된다.

위와 같이 직접 수식을 만들어도 되고, scikit-learn 에서 제공하는 MinMaxScaler 를 써도 된다.

```
scaler = MinMaxScaler()
scaler.fit(df)
scaled = scaler.transform(df)
```

## 5.3 어떤 스케일러를 쓸까

스케일러 선택 가이드

- |—— 정규분포에 가까움 → StandardScaler
- |—— 0~1 범위가 의미 있음 (이미지 픽셀 등) → MinMaxScaler
- |—— 이상치가 많음 → RobustScaler
- |—— 트리 기반 모델 (RF, GBM, XGBoost) → 스케일링 불필요

중요: 스케일링은 반드시 **훈련 데이터의 통계로 fit** 하고, 테스트 데이터는 **transform** 만 한다. 그렇지 않으면 "테스트 데이터의 정보가 훈련에 새어 들어가는(데이터 누수)" 문제가 생긴다. 사실 테스트데이터는 훈련과정의 입장에서 보면 '존재하지 않는' 데이터이기 때문이다.

트리기반 모델은 스케일링이 불필요하다는 점도 기억해 두자.

## 6. 범주형 변수 인코딩

"male/female", "S/C/Q" 같은 문자열 카테고리는 모델이 직접 다룰 수 없다. 숫자로 변환해야 하는데, 그 방식이 두 가지로 나뉜다.

### 6.1 Label Encoding

각 카테고리에 정수 라벨을 부여한다(female=0, male=1). 코드는 간단하지만, 모델이 "male 이 female 보다 1 만큼 크다"는 오해를 할 수 있다. 따라서 순서가 의미 있는 "순서형 변수(ordinal)"에만 안전하다(예: 학력: 초졸 < 중졸 < 고졸 < 대졸).

```
from sklearn.preprocessing import LabelEncoder
items = ['TV', 'Refrigerator', 'computer', 'fan', 'blender']
encoder = LabelEncoder()
encoder.fit(items)
labels = encoder.transform(items)
labels
```

### 6.2 One-Hot Encoding

각 카테고리를 0/1 더미 변수로 펼친다. 이 방식이 일반 범주형 변수의 **표준**이다.

예를 들면 다음과 같다.

원본 컬럼: embarked

S  
C  
Q  
S

One-Hot 인코딩 결과:

| emb_S | emb_C | emb_Q |
|-------|-------|-------|
| 1     | 0     | 0     |
| 0     | 1     | 0     |
| 0     | 0     | 1     |
| 1     | 0     | 0     |

```
from sklearn.preprocessing import OneHotEncoder
items = ['TV', 'Refrigerator', 'computer', 'fan', 'blender']
# 2 차원 array 로 변환
items = np.array(items).reshape(-1, 1)
oh_encoder = OneHotEncoder()
oh_encoder.fit(items)
oh_labels = oh_encoder.transform(items)
oh_labels.toarray()
```

일반적으로 가장 많이 사용하는 것은 pandas 에서 원핫 인코딩을 위해 제공하는 `get_dummies()` 함수를 사용하는 것이다.

```
import pandas as pd
df = pd.DataFrame({'item': ['TV', 'Refrigerator', 'computer', 'fan', 'blender']})
pd.get_dummies(df)
```

### 6.3 카테고리 수가 매우 많은 경우(고차원 폭발)

- 도시 이름처럼 1,000 개 이상의 카테고리 → One-Hot 은 1,000 개 컬럼 생성, 비효율.
- 대안 1: 빈도 기반으로 상위 N 개만 남기고 나머지는 "Other"로 합치기.
- 대안 2: Target Encoding — 해당 카테고리에서의 타겟 평균값으로 치환.
- 대안 3: 임베딩(딥러닝) — 학습 가능한 저차원 벡터로 표현.

## 7. 사이킷런 코드 실습 — Titanic 전처리

seaborn 에 내장된 Titanic 데이터셋을 이용한다. 891 명 승객의 생존(0/1)을 예측하는 이진 분류 문제로, 위에서 언급한 모든 전처리 항목을 한 번에 다룰 수 있다.

### 실습 환경 준비

```
pip install scikit-learn pandas seaborn matplotlib
```

### [1 단계] 데이터 로드와 전반 점검

```
import pandas as pd, seaborn as sns

df = sns.load_dataset("titanic")
print(df.shape)      # (891, 15)
print(df.dtypes)
print(df.isnull().sum().sort_values(ascending=False).head(8))
```

결측은 deck(688), age(177), embarked(2) 정도다. deck 은 결측이 너무 많아 컬럼째 빼는 편이 낫고, age 는 대체가 적절하다.

### [2 단계] 사용할 컬럼만 추리고 결측치 처리

```
cols = ["survived", "pclass", "sex", "age",
        "sibsp", "parch", "fare", "embarked"]
df = df[cols].copy()

# age: 성별·등급별 중앙값으로 대체 (단순 평균보다 합리적)
df["age"] = df.groupby(["sex", "pclass"])["age"].\
    .transform(lambda s: s.fillna(s.median()))
```

```
# embarked: 최빈값으로 대체
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])

print(df.isnull().sum())
```

### [3 단계] 이상치 진단 — fare(운임)의 박스플롯

```
import matplotlib.pyplot as plt

plt.boxplot(df["fare"])
plt.title("Fare boxplot")
plt.show()

q1, q3 = df["fare"].quantile([0.25, 0.75])
iqr = q3 - q1
high = q3 + 1.5 * iqr
print("이상치 임계값(상한):", high)
print("이상치 개수:", (df["fare"] > high).sum())
```

운임은 오른쪽으로 매우 치우친 분포로, 일부 1 등실 승객의 운임이 매우 높다. 이런 분포는 로그 변환을 적용하면 한층 다루기 쉬워진다.

```
import numpy as np
df["fare_log"] = np.log1p(df["fare"]) # log(1 + x)
```

### [4 단계] 범주형 인코딩 — One-Hot

```
df_enc = pd.get_dummies(df, columns=["sex", "embarked"], drop_first=True)
# drop_first=True : 더미 변수 함정 방지 (k 개 카테고리 → k-1 개 컬럼)
print(df_enc.head())
print(df_enc.columns.tolist())
```

### [5 단계] 입력/출력 분리, 훈련/테스트 분리

```
from sklearn.model_selection import train_test_split

y = df_enc["survived"]
X = df_enc.drop(columns=["survived"])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

### [6 단계] 스케일링 — 표준화

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
```

```
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)      # 절대 fit_transform 쓰지 않기
```

## [7 단계] 전처리 전후 모델 정확도 비교

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# (a) 스케일링 없이
knn1 = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train)
acc1 = accuracy_score(y_test, knn1.predict(X_test))

# (b) 스케일링 후
knn2 = KNeighborsClassifier(n_neighbors=5).fit(X_train_s, y_train)
acc2 = accuracy_score(y_test, knn2.predict(X_test_s))

print(f"스케일링 X : {acc1:.4f}")
print(f"스케일링 O : {acc2:.4f}")
```

KNN 처럼 거리 기반 모델에서는 스케일링만으로도 정확도가 5~10%p 가까이 오르는 경우가 흔하다.

## [8 단계] sklearn Pipeline — 전처리를 "한 줄"로 묶기

위 단계들은 데이터 누수를 막기 위해 train/test 에 따로 적용해야 한다. Pipeline 은 이 흐름을 자동화하고 교차검증·GridSearch 와도 그대로 연결된다.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

num_cols = ["age", "sibsp", "parch", "fare"]
cat_cols = ["sex", "embarked"]

pre = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(drop="first", sparse_output=False), cat_cols),
], remainder="passthrough")

pipe = Pipeline([
    ("pre", pre),
    ("clf", KNeighborsClassifier(n_neighbors=5)),
])

pipe.fit(X_train_raw, y_train)      # X_train_raw 는 전처리 전 원본
print("파이프라인 정확도:", pipe.score(X_test_raw, y_test))
```

Pipeline 은 fit() 시 훈련 데이터로만 통계를 학습하고, predict()/score() 시 테스트 데이터에 transform 만 적용하므로 데이터 누수가 원천 차단된다.

## 8. 오늘 수업 정리

- 실제 데이터에는 결측치, 이상치, 단위 차이, 범주형 변수가 섞여 있어 전처리가 필수다.
- EDA 는 dtypes/info/describe 와 박스플롯·히스토그램으로 데이터의 "성격"을 파악하는 작업이다.
- 결측치는 비율과 패턴에 따라 삭제/대치/IsMissing 더미 추가 등 전략을 달리한다.
- 이상치는 IQR 규칙으로 검출하고, 모델 특성에 따라 제거·원저화·로그변환을 선택한다.
- StandardScaler(z-score)와 MinMaxScaler(0~1) 중 데이터의 분포 형태에 맞는 것을 고른다. 트리 기반 모델은 스케일링이 필요 없다.
- 범주형 변수는 보통 One-Hot 인코딩이 안전하다. 카테고리가 매우 많으면 빈도 기반 묶기·Target Encoding 을 검토한다.
- 훈련/테스트 데이터의 통계가 섞이지 않도록 fit 은 train 에서만, transform 은 test 에 — 이 흐름을 자동화하는 도구가 sklearn Pipeline 이다.

## 다음 시간 예고

4 교시는 "결정트리(Decision Tree) 심화"다. 정보 이득과 지니 계수, 트리의 깊이/잎 노드 제약, 가지치기까지 다루고, 학습한 트리를 직접 시각화해 "왜 그렇게 예측했는지" 해석한다.

## 과제

- Titanic 데이터에서 age 결측을 (a) 전체 평균 (b) 성별×등급별 중앙값 (c) 결측 자체를 새로운 카테고리(IsAgeMissing)로 처리 — 세 가지 방법으로 처리하고 KNN 정확도를 비교한다.
- fare 컬럼에 (a) 원본 (b) 로그변환 (c) 원저화(상위 1% 클리핑) 세 가지를 적용해 정확도와 ROC-AUC 를 함께 비교한다. 어느 방법이 가장 좋았는지 그 이유를 두 줄로 적어라.
- seaborn.load\_dataset("diamonds")의 cut/color/clarity 세 컬럼을 (a) Label Encoding (b) One-Hot Encoding 으로 변환했을 때 LinearRegression 으로 가격을 예측한 결과( $R^2$ )가 어떻게 다른지 비교 보고서를 짧게 작성한다.

## 이미지 출처

- [그림 0] RMS\_Titanic\_3.jpg — F.G.O. Stuart, 1912 (Public Domain), Wikimedia Commons
- [그림 1] Box\_Plot\_Picture.png — Wikimedia Commons
- [그림 2] Boxplot\_vs\_PDF.svg — Jhguch / Chen-Pan Liao, Wikimedia Commons (CC BY-SA 2.5)
- [그림 3] Standard\_deviation\_diagram.svg — Mwtoews, Wikimedia Commons (CC BY 2.5)